
**Large-Scale Design & Analysis of Neural
Networks
System Test Plan
Version <2.0>
12/5/2024**

Document Control

Distribution List

The following list of people will receive a copy of this document every time a new version of this document becomes available:

Teaching assistants:

Clay Pate

Customer(s):

M. Ilhan Akbas

John M. Thompson

Project team members:

Daniel Machado

Simon Cole

Muhammed Sharif Adepetu

Leah Powell

Change Summary

The following table details changes made between versions of this document:

Version	Date	Modifier	Description
1.1	12/3/2024	Leah Powell	Document Creation, 1.1, 1.2, 1.3, 5.2, 3
1.2	12/4/2024	Muhammed-Sharif	1.4
1.3	12/5/2024	Daniel Machado	2.4, 2.5 + Added 7 UI Test Cases
1.4	12/5/2024	Leah Powell	4.2, 5.1, 2.4, 2.5, 2.6
1.5	12/5/2024	Muhammed-Sharif	Fleshed out test cases / Formatting
1.6	12/5/2024	Simon Cole	Added/edited test cases
1.61	1/7/2025	Muhammed-Sharif Adepetu	Responded to comments from last semester
1.7	4/10/2025	Muhammed-Sharif Adepetu	Removed / Edited certain test cases. Filled out completed tests
1.75	4/14/2025	Muhammed-Sharif Adepetu	Created more test cases according to system requirements
1.8	4/15/2025	Leah Powell	Added test cases 13-23, updated matrix
1.9	4/18/2025	Muhammed-Sharif Adepetu	Conducted tests with package.

2.0	4/23/2025	Leah	Edited test cases, conducted test cases, updated test plan table
-----	-----------	------	---

Table of Contents

Document Control	ii
Distribution List.....	ii
Change Summary	ii
1. Introduction	1
1.1. Purpose	1
1.2. Scope	1
1.3. System Overview	1
1.4. Testing Approach Overview	1
1.5. Testing Entrance Criteria	2
1.6. Document Overview.....	3
1.7. References	3
2. Testing Approach	4
2.1. Testing Types	4
2.1.1. Usability Testing	4
2.1.2. Functionality Testing	4
2.2. Testing Suspension Criteria and Resumption Requirements	4
2.2.1. Suspension Criteria.....	4
2.2.2. Resumption Requirements	4
2.3. Testing Environment	4
2.4. Testing Assumptions	4
2.5. Testing Risks and Contingencies.....	5
2.6. Test Plan	6
3. Test Schedule	6
4. Traceability Matrix and Defect Tracking	8
4.1. Traceability Matrix.....	8
4.2. Defect Severity Definitions.....	8
5. Test Cases	10
5.1. Test Case 1	10
5.2. Test Case 2	11
5.3. Test Case 3	12
5.4. Test Case 4	13
5.5 Test Case 5	14
5.6 Test Case 6	16

5.5. Test Case 7	17
5.8 Test Case 8	18
5.9 Test Case 9	19
5.10 Test Case 10	20
5.11 Test Case 11	21
5.12 Test Case 12	22
5.13 Test Case 13	24
5.14 Test Case 14	25
5.15 Test Case 15	26
5.16 Test Case 16	27
5.17 Test Case 17	28
5.18 Test Case 18	29
5.19 Test Case 19	30
5.20 Test Case 20	31
5.21 Test Case 21	32
5.22 Test Case 22	33
5.23 Test Case 23	34

1. Introduction

1.1. Purpose

The project's goal is to implement a pipeline for large-scale design, creation, and analysis of Bayesian Neural Networks. These networks are using a newer, more modern approach that uses Baye's Theorem for weight distributions. The purpose of this document is to outline a detailed test plan that describes our approach, schedule, and overall organization of testing. The document first covers testing approaches. This includes but is not limited to testing for risks, testing for functionality, and laying out the actual test plan. This document also outlines the test schedule, and actual test cases. Lastly, it covers traceability matrix and defect tracking.

1.2. Scope

The scope of the project includes the release of a github repo for download in order for the user to use the pipeline program.

1. Prototype Release (Version 0.1):

- a. Initial implementation of neural network architecture(s) under study.
- b. Basic data preprocessing pipeline.
- c. Preliminary experiments to validate setup.
- d. Disjointed classes with basic functionality

2. First Analysis Release (Version 0.5):

- a. Results from experiments focusing on the probability distribution of weights for baseline architectures.
- b. Visualizations of weight distributions and their evolution across training epochs.
- c. Comparative study of different activation functions and their impact on weight distributions.

3. Optimization Release (Version 1.0):

- a. Inclusion of optimizations such as alternative training strategies or network modifications (e.g., increasing the number of neurons, changing layer types).
- b. Refined tools for weight analysis.
- c. Comprehensive results and insights into unique patterns in weight distributions.

1.3. System Overview

The system is organized into modules that meet the high-end goals of the project. The modules cover Network Loading, Loss Analysis Tools, Weight Binning & Visualization, Success Criterion Evaluation, and Export/Reporting. The system does not implement any external interfaces as it is a software-based project, organized into individual files that handle all the modules.

1.4. Testing Approach Overview

The testing approach for the neural network analysis project follows a structured, multilevel methodology to ensure robust evaluation of both functionality and performance. This overview links directly to the conceptual diagram outlined in Section 1.3, which depicts the project's components and interactions.

1.4.1.1. Objective and Scope of Testing Levels

1. Unit Testing (Level 1):

- a. **Objective:** Validate the correctness of individual components, such as neural network layers, activation functions, and data preprocessing steps.
- b. **Scope:** Includes testing layer outputs for expected behavior, ensuring accurate implementation of custom functions, and verifying that data preprocessing maintains data integrity.

2. Integration Testing (Level 2):

- a. **Objective:** Ensure seamless interaction between components such as data pipelines, model training modules, and visualization tools.
- b. **Scope:** Covers end-to-end data flow, interaction between training and evaluation phases, and compatibility of third-party libraries.

1.5. Testing Entrance Criteria

To implement system testing, these requirements must be met for testing readiness:

Environment Setup:

- A fully configured testing environment, replicating the production setup as closely as possible, must be available.
- All necessary hardware, software, and network configurations should be in place and verified.
- Access to necessary tools for logging, debugging, and monitoring system performance should be ensured.

Stable Build Availability:

- The system built intended for testing must be stable and deployed to the testing environment.
- All components and modules required for end-to-end testing should be integrated and functional.

Test Data:

- Well-prepared test data sets must be available and validated for use.
- Test data should include edge cases, boundary values, and scenarios for positive and negative testing.

1.6. Document Overview

1.7. References

S. Adepetu, S. Cole, D. Machado, and L. Powell, *Project Proposal Document*. 2024.

S. Adepetu, S. Cole, D. Machado, and L. Powell, *Software Design Documentation*. 2024.

S. Adepetu, S. Cole, D. Machado, and L. Powell, *System Requirements Specification*. 2024

2. Testing Approach

2.1. Testing Types

2.1.1. Usability Testing

Ensure that the required files in the program package are tested for accuracy and general usability. The goal of this testing is to ensure that the files will run without error or unexpected behavior. The user should be able to navigate from file to file without any confusion or issue.

2.1.2. Functionality Testing

Functionality testing will ensure adherence to the project outlined in the Software Design Document and System Requirement Specifications. The functionality of each module in the system shall be carried out in the runnable program files.

2.2. Testing Suspension Criteria and Resumption Requirements

This section will specify the criteria that will be used to suspend all or a portion of the testing activities on the items associated with this test plan.

2.2.1. Suspension Criteria

Testing will be suspended if the incidents found will not allow further testing of the system/application under-test. If testing is halted, and changes are made to the hardware, software, or database, it is up to the Testing Manager to determine whether the test plan will be re-executed, or part of the plan will be re-executed.

2.2.2. Resumption Requirements

2.3. Testing Environment

The testing environment for this plan is configured to ensure reproducibility, consistency, and compatibility that includes:

- The following test cases will be executed in an environment that runs on Windows.
- The tests will be run in a Jupiter notebook, and/or

2.4. Testing Assumptions

- There is a pre-built Neural Network Architecture
- There is a pre-processed dataset, i.e. the data is free from missing values and contains normalized data types
- The processed data fits the Neural Network architecture we are using

- Testing environments are compatible
 - The environment software/hardware is sufficient enough to support the necessary components and libraries.
- All libraries and framework versions used are compatible and behave as expected in the environment

2.5. Testing Risks and Contingencies

The risks and contingencies possible for our project are:

1. Data Generation risks – *data is checked, validated, and formatted before it is used*
 - a. improper Data Generation
 - b. issues or Deviations in Input Features
2. Neural Network Generation Risks – *perform preliminary testing with smaller datasets for tuning hyperparameters*
 - a. failure to converge
 - b. dataset noise
3. Excessive Model Failure – *set a reasonable maximum number of attempts*
4. Pipeline Failure / Unexpected Inputs – *include input validation at the appropriate stages of the pipeline and generation, use edge case datasets for testing.*

2.6. Test Plan

Table 1: Test Plan

ID	Test	Status	Date	Notes
1	Verify .csv file Created for Training	Closed	10 Apr-25	N/A
2	Initializing Network	Closed	10 Apr-25	N/A
3	Convergence Threshold Input Validation	Closed	10 Apr-25	N/A
4	Successful loss customization	Closed	10 Apr-25	N/A
5	Data Loader Input	Closed	10 Apr-25	N/A
6	Loss Function input	Closed	10 Apr-25	N/A
7	Automatic Generation of k Networks	Closed	10 Apr-25	N/A
8	Saving Neural Network	Closed	10 Apr-25	N/A
9	Weight Binning Initialization	Closed	10 Apr-25	N/A
10	Single-Peak Gaussian verification	Closed	18 Apr-25	N/A
11	Sparse Data Handling	Closed	18 Apr-25	N/A
12	Weight histogram display verification	Closed	18 Apr-25	N/A
13	File Path Validation			
14	Bin Edge Labeling	Closed	18 Apr-25	N/A
15	Bin Frequency Population	Closed	22 Apr-25	N/A
16	Walkthrough Strategy Verification	Closed	22 Apr-25	N/A
17	Successful network Verification	Closed	22 Apr-25	N/A
18	Gaussian & GMM fit Verification	Closed	22 Apr-25	N/A
19	Unique Distribution(s) Verification	Closed	22 Apr-25	N/A
20	Network Loss Verification	Closed	22 Apr-25	N/A
21	Test Network Generation Count	Closed	22 Apr-25	N/A
22	Verify Max Attempt Threshold	Closed	22 Apr-25	N/A
23	Loss Reset Verification	Closed	22 Apr-25	N/A

3. Test Schedule

Testing Schedules: test plans are carried out at every other sprint, and this document is to be managed and written to at least once every sprint keep test plans and test plan results up to date.

4. Traceability Matrix and Defect Tracking

4.1. Traceability Matrix

Table 2: Traceability Matrix

Req. ID	Test Case(s)
<i>STIM1</i>	17, 18, 7
<i>STIM1.2</i>	17, 18, 7
<i>STIM1.4</i>	27
<i>STIM2</i>	20, 23, 24
<i>STIM2.2</i>	19, 24, 8
<i>STIM2.3</i>	23, 25
<i>STIM2.4</i>	28
<i>STIM3.3</i>	16, 22
<i>STIM3.4</i>	20,
<i>SI1</i>	15
<i>SI3</i>	27
<i>RF1</i>	9
<i>DR1</i>	15
<i>DR2</i>	15
<i>DR3</i>	19
<i>F4</i>	21, 26
<i>F5</i>	16, 20, 24,
<i>F7</i>	10
<i>O1</i>	17, 7
<i>O2</i>	19
<i>UI1</i>	27,2
<i>UI2</i>	3
<i>UI3</i>	4
<i>UI4</i>	5
<i>UI5</i>	21
<i>UI8</i>	1
<i>PR3</i>	20
<i>QR1</i>	12

4.2. Defect Severity Definitions

Critical	The defect causes a catastrophic or severe error that results in major problems and the functionality is rendered unavailable to the user. A manual procedure is impossible to be implemented, or high effort is required to remedy the defect. Examples of critical defects are as follows: • System abnormally terminates • Data cannot flow through a business function/lifecycle • Data is corrupted or cannot post to the database
----------	--

Medium	The defect that does not seriously impair system function can be categorized as a medium defect. A manual procedure requiring medium effort can be implemented to remedy the defect. Examples of medium defects are as follows: • Form navigation is incorrect • Field labels are not consistent with global terminology
Low	The defect is cosmetic or has little to no impact on system functionality. A manual procedure requiring low effort can be implemented to remedy the defect. Examples of low defects are as follows: • Repositioning of fields on screens • Text font on reports is incorrect

5. Test Cases

5.1. Test Case 1

Objective: properly create .csv file for training

Notes: This test is conducted by running the generate Data script, at which the system shall create a to be saved into a .csv file for training of the neural network, which is essential for all subsequent tests.

Test No.: 1		Current Status: Completed		
Test title: Verify that .csv file is created correctly for training				
Testing approach: Black box testing				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Verify that the environment contains NumPy and pandas installed.	Code will not function without these.	The system shall create a fully usable .csv file for training data.	
2	Run the script for GenerateData.py.	This is the file that creates the .csv file.	The code shall run successfully.	
3	Check the correct directory for the "SimpleReg.csv" file.	The file needs to be in the right folder.	The file shall exist in the directory with the name "SimpleReg.csv"	
4	Verify the file contains three columns of (a, b, y).	These are columns used for tensors.	The file shall contain those three columns.	
5	Verify (a, b) contain floating point values and y contains a computed value using the target function.	This is a standard for tensors and matrices in NN training data.	The left columns a and b shall contain floating point values, and the corresponding y column shall be correct according to the target function.	

<i>Concluding Remarks: Csv file successfully created and saved to correct location</i>	
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>	<i>Date Completed:</i> <i>Apr 10 2025</i>

5.2. Test Case 2

Objective: Initialize 'NetworkAnalyzer' Class

Notes: Ensure that the package is correctly installed by attempting to initialize the NetworkAnalyzer Module with all required attributes. Test for proper assignment of values and verify no initialization errors occur

Test No.: 2		Current Status: Completed		
Test title: Initializing network				
Testing approach: Instantiate the class, check attribute values, confirm no errors during initialization. Validate that default or input values are correctly assigned. UI1.				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Class should accept all the initialization parameters	Instantiate the class that will be used to perform analysis	Class shall be instantiated	
2	Verify attribute values	Check if the attributes match expected values	Attributes are set to the correct values (either default or input).	All attributes

3	<i>Check for errors during initialization</i>	<i>Ensure no initialization errors occur</i>	<i>No exceptions or errors are raised during class instantiation.</i>	<i>No errors thrown</i>
<i>Concluding Remarks: Class initialized without any issue. It is important for all requirements to be installed before attempting.</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed:</i> <i>10 April 2025</i>	

5.3. Test Case 3

Objective: Validate Convergence threshold.

Notes: Ensure that the network converges within the expected number of epochs and does not oscillate. Verify that the training process leads to an acceptable error rate.

<i>Test No.: 3</i>	<i>Current Status: Completed</i>
<i>Test title: Convergence Threshold Input Validation</i>	
<i>Testing approach: Input a variety of thresholds to observe system behavior. UI2.</i>	

<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
<i>1</i>	<i>Input valid convergence threshold</i>	<i>Confirm acceptance and usage</i>	<i>System uses input during training and stops training particular network once convergence</i>	
<i>2</i>	<i>Input Negative or Zero value</i>	<i>Test for invalid threshold values</i>	<i>System returns warning</i>	
<i>3</i>	<i>Omit threshold input</i>	<i>Test default behavior</i>	<i>System applies default convergence value.</i>	<i>System will set convergence threshold to 1000000 so it always works</i>
<i>Tests passed</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>		<i>Date Completed:</i> <i>10 April 2025</i>		

5.4. Test Case 4

Objective: Ensure user can customize the success loss.

Notes: Ensure the network generation process correctly initializes all parameters, including weights, biases, and activation functions. Validate that the network is ready for training without errors.

Test No.: 4		Current Status: Completed		
Test title: Success loss customization				
Testing approach: Input various loss thresholds and monitor behavior. UI3.				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Input a valid success loss value	Validate system stores & uses value	System uses value for evaluating success	
2	Input an invalid (non-numeric) value	Test error handling	System returns error or prompts correction	
3	Run training with different success losses	Confirm impact on training result	The loss displayed matches the expected result based on network output.	
Sometimes the system will end warly				
Testing Team:		Date Completed:		
Muhammed-Sharif Adepetu		10 April 2025		

5.5 Test Case 5

Objective: Ensure the system accepts data loader input.

Notes: NA

Test No.: 5			Current Status: Completed	
Test title: data loader input				
Testing approach: Supply different format for data loaders. UI4.				
STEP	OPERATOR ACTION	PURPOSE	EXEPCTED RESULTS	COMMENTS
1	Provide valid training loader	Ensure loader is processed	System loads training data	
2	Input invalid data loader format	Test for input validation	System displays appropriate error	
3	Input empty loader	Test for input validation	System displays error	
Concluding Remarks: Ensuring data loaders are working correctly is essential for system to properly work				
Testing Team: Muhammed-Sharif Adepetu			Date Completed: 10 Apr 2025	

5.6 Test Case 6

Objective: Ensure system correctly uses loss function.

Notes: Ensure that the Network analyzer module accepts and accurately calculates the loss of the network after each epoch

Test No.: 6			Current Status: Completed	
Test title: Loss Function input				
Testing approach: Provide valid/invalid loss functions. UI5.				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Select valid / relevant PyTorch loss function	Verify loss function is accepted and displays function to user	System uses selected loss function	
2	Leave input black	Test error handling	Class is unable to initialize. System returns an error	

<i>Concluding Remarks: << Filled in by the person who completed the test >></i>	
<i>Testing Team:</i> <i>Muhammed-Shaif Adepetu</i>	<i>Date Completed:</i> <i>10 April 2025</i>

5.5. Test Case 7

Objective: Verify automatic generation of k neural networks.

Notes:

Test No.: 8			Current Status: Completed	
Test title: Automatic Generation of k Networks				
Testing approach: Generation and saving of k neural networks.				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Set number of networks to produce to be valid positive integer (10)	Testing standard generation	The system creates 5 distinct neural networks	

2	Input 0 for number of networks	Validate edge-case handling	No networks generated	
3	Input non-integer value for k	Test input type enforcement	System throws error. No networks are generated	
<i>Networks generated</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed:</i> <i>10 April 2025</i>	

5.8 Test Case 8

Objective: Ensure the system saves neural networks only if they meet the user-defined loss threshold. Validate threshold configuration and saving functionality

Notes:

<i>Test No.: 8</i>	<i>Current Status: Completed</i>
<i>Test title: Saving Neural Network's Based on Loss Threshold</i>	
<i>Testing approach: Test Saving functionality by training NNs, comparing their loss to the threshold, and verifying only eligible networks are saved.</i>	

<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	Train the neural network	Perform training to calculate network loss	The network is trained, and the loss value is computed.	
2	Compare network loss with the threshold	<i>Check if the loss meets the user-defined threshold</i>	Networks with a loss below or equal to the threshold are eligible for saving.	
3	Save networks that meet the threshold	Store the network if it passes the threshold test	System saves .pt file of networks that meet or exceed the loss threshold.	
<i>Concluding Remarks: NA</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed: 10 April 2025</i>	

5.9 Test Case 9

Objective: Initialize 'Weight Binnig' Class

Notes: Ensure that the package is correctly installed by attempting to initialize the Weight Binning Module with all required attributes. Test for proper assignment of values and verify no initialization errors occur

<i>Test No.: 9</i>	<i>Current Status: Completed</i>
--------------------	----------------------------------

<i>Test title: Initializing weight binning</i>				
<i>Testing approach: Instantiate the class, check attribute values, confirm no errors during initialization. Validate that default or input values are correctly assigned</i>				
<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
<i>1</i>	<i>Class should accept user-defined</i>	<i>Instantiate the class that will be used to perform analysis</i>	<i>Class shall be instantiated</i>	
<i>2</i>	<i>Class loads the path of saved networks</i>		<i>Class shall be instantiated</i>	<i>Module should be initialized using the same architecture that networks were trained on</i>
<i>3</i>	<i>Verify attribute values</i>	<i>Check if the attributes match expected values</i>	<i>Attributes are set to the correct values (either default or input).</i>	
<i>4</i>	<i>Check for errors during initialization</i>	<i>Ensure no initialization errors occur</i>	<i>No exceptions or errors are raised during class instantiation.</i>	<i>No errors thrown</i>
<i>Concluding Remarks: Class initialized without any issue. It is important for all requirements to be installed before attempting.</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>		<i>Date Completed:</i> <i>10 April 2025</i>		

5.10 Test Case 10

Objective: Verify Single-Peak Gaussian

Notes: Ensure that the model can creates a set of histograms with single-peak gaussian distributions

<i>Test No.: 10</i>	<i>Current Status: Completed</i>
---------------------	----------------------------------

<i>Test title: Single-Peak Gaussian verification</i>				
<i>Testing approach: Test the model with increasing dataset sizes and monitor performance metrics such as training time and accuracy,</i>				
<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	Initialize Weight Bining module using a set of	Start with small dataset to establish a baseline	The model should train quickly and with expected accuracy	
2	Increase dataset incrementally	Gradually feed larger datasets to model	Training time should increase proportionally without significant slowdowns	
3	Monitor performance	Track training time, accuracy and resource usage	The model should maintain performance even with larger datasets	
<i>Concluding Remarks: NA</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed: 18 Apr 2025</i>	

5.11 Test Case 11

Objective: Validate Model's Ability to Handle Sparse Data

Notes: Ensure that the model performs well when the input data contains significant sparsity

<i>Test No.: 11</i>	<i>Current Status: Completed</i>
<i>Test title: Sparce Data Handling</i>	

<i>Testing approach: Provide sparse datasets to the model and evaluate its ability to learn and make accurate predictions.</i>				
<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	Prepare sparse dataset	Create datasets with high percentage of zero values	Sparse datasets should not cause errors during training or evaluation	
2	Train the model on sparce data	Feed sparse data to model during training	The model should be able to process sparse data efficiently without degradation in performance	
3	Evaluate prediction accuracy	Test the model's accuracy on sparse validation data	Accuracy should remain consistent despite sparsity in input data	
<i>Concluding Remarks: prediction accuracy significantly impaired. Gaussian distributions vary drastically from models predicted on more substantial data</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed: Apr 18 2025</i>	

5.12 Test Case 12

Objective: Ensure the system correctly displays the weight histogram for a selected layer.

Notes:

<i>Test No.: 12</i>	<i>Current Status: Completed</i>
<i>Test title: Weight histogram display verification</i>	

<i>Testing approach: Manually call plot_distribution method in specific layer and visually verify weight histogram is correctly rendered</i>				
<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	Select sepcific layer (eg layer 2)	To test if the system correctly identifies the layer for analysis.		
2	Run <i>Plot_distribution</i>	verify the output visualization is generated correctly	Histogram for the selected layer is rendered accurately.	
<i>Concluding Remarks: Class acts accordingly</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed: 18 Apr 2024</i>	

5.13 Test Case 13

Objective: *Ensure validation and retrieval of file paths for configuration files, data files, and model output directories.*

Notes:

Test NO: 13			Current Status: Completed	
Test title: File Path Validation				
Testing approach: Ensure system properly validates file paths for data files and model output directories				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Download package	Need package to test file pathway	No errors, no missing dependencies or libraries	
2	Verify valid file path exists using cmd	Path needs to exist to test it works in code	Pathway found, no error or missing file path	
3	Input file pathway in each respective zone within package and run files till completion		Files run, new pngs are saved to output folder and networks are saved to networks folder	
Concluding Remarks:				

<i>Testing Team: Muhammed-Sharif Adepetu</i>	<i>Date Completed: 18 Apr 2025</i>
--	------------------------------------

5.14 Test Case 14

Objective: Check that histogram x-axis uses correct bin edges.

Notes:

Test No.: 14			Current Status: Completed	
Test title: Bin Edge Labeling on Histogram				
Testing approach: Functionality testing				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Provide array of bin_edges (e.g., [0, 0.5, 1.0])	Simulate expected label values.		
2	Generate weight histogram			
3	Inspect histogram labels for accuracy	validate labeling reflects input edges.	X-axis is labeled using actual bin edge values.	

<i>Concluding Remarks: NA</i>	
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>	<i>Date Completed: 18 Apr 2024</i>

5.15 Test Case 15

Objective: Ensure correct bin population with weight occurrences.

Notes:

Test No.: 15			Current Status: Completed	
Test title: Bin Frequencies Population				
Testing approach: Verify weight values are accurately assigned to correct bins based on specified bin edges				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Initialize weight binning class with dataset of multiple small neural networks			

2	Run binning and accumulation logic		I	
3	Verify bin counts match expected values	check frequency correctness.	Bins are populated with correct count totals.	
<i>Concluding Remarks: NA</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu,</i>			<i>Date Completed: 22 April 2025</i>	

5.16 Test Case 16

Objective: Ensure that weights walkthrough strategy is correctly implemented

Notes:

Test No.: 16			Current Status: Completed	
Test title: Walkthrough Strategy Verification				
Testing approach: Validating correct implementation of walkthrough by checking if the system correctly iterates though layers				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Run <i>weights_walkthrough</i>	Initiate traversal so we can view all Neuron weight distributions through the network	Walkthrough starts	
2	Verify order of neuron traversal	<i>Confirm Walkthrough completed correctly</i>	Images of all weights are	

			displayed in correct order	
<i>Concluding Remarks: NA</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed: 22 April 2025</i>	

5.17 Test Case 17

Objective: Network success validation

Notes:

Test No.: 17			Current Status: Completed	
Test title: Successful Network Verification				
Testing approach: Generate networks then test if they are successful under pre-defined performance metrics				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Run <i>setup()</i> automatically or manually within the testing.ipynb file	Prepare Testing Data	Data loaded with no errors	
2	Run <i>test_is_successful_true()</i>	Check network marked successful with low enough loss	Images of all weights are displayed in correct order	
Concluding Remarks: NA				
Testing Team: Muhammed-Sharif Adepetu,			Date Completed: 22 April 2025	

5.18 Test Case 18

Objective: Verify fit_distributions method correctly fits either single Gaussian or GMM distributions to normalized weight histograms

Notes:

Test No.: 18			Current Status: Completed	
Test title: Gaussian & GMM fit Verification				
Testing approach: Functionality Testing. Testing fit_distributions method to ensure it accurately fits either single gaussian of GMM models to normalized weight histograms for each weight connection				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Prepare mock normalized weight distributions	Prepare Testing Data	Data loaded with no errors	
2	Run <i>fit_distributions()</i> <i>With false multi peak</i>	Test the single Gaussian fitting branch	fit_params initialized with valid Gaussian parameters	

3	Run <i>fit_distributions()</i> <i>With true multi peak and 2 peaks pr layer</i>	Test the GMM fitting process using a known random seed for reproducibility	gmm_models is initialized, and each GMM has 2 components as specified	
4	Provide empty or zeroed histograms for weight inputs	Validate fallback mechanism when data is insufficient for fitting	system should issue a warning and not return anything	
<i>Concluding Remarks: System works as expected</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed: 22 April 2025</i>	

5.19 Test Case 19

Objective: Verify plot_unique_distributions correctly identifies and visualizes one distribution per cluster in a specified layer.

Notes:

Test No.: 19			Current Status: Completed	
Test title: Unique Distribution(s) Verification				
Testing approach: Functionality Testing.				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Run plot_unique_distributions(indices, layer=0, multi_peak=False)	Check the function visualizes only one distribution per unique cluster index	Data loaded with no errors	
2	Repeat step one with	Validate same functionality	One histogram + GMM plot	

	<i>multi_peak=True using fit_distributions(multi_peak=True) and cluster_distributions(multi_peak=True)</i>	for GMM- based clustering	for each unique cluster in the specified layer; console prints GMM cluster IDs and locations	
3	<i>plot_unique_distributions</i> without calling <i>fit_distributions</i> beforehand	Verify that the method catches the absence of fitted models and exits with a warning	No plots created. Console alerts user and tells them to call fit distributions first	
<i>Concluding Remarks: NA</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed: 22 April 2025</i>	

5.20 Test Case 20

Objective: Network loss validation

Notes:

<i>Test No.: 20</i>	<i>Current Status: Completed</i>
<i>Test title: Network Loss Verification</i>	
<i>Testing approach: Functionality testing, black box testing, REQ-STIM3.3, F5</i>	

STEP	OPERATOR ACTION	PURPOSE	EXEPCTED RESULTS	COMMENTS
1	Run <i>setup()</i> automatically or manually within the testing.ipynb file	Prepare Testing Data	Data loaded with no errors	
2	Run <i>test_is_successful_true()</i>	Check network marked successful with low enough loss	Returns true	
Concluding Remarks: NA				
Testing Team: Muhammed-Sharif Adepetu, Leah Thomas,			Date Completed: 22 April 2025	

5.21 Test Case 21

Objective: Specific count Network generation validation

Notes:

Test No.: 21	Current Status: Completed
Test title: Test network generation count	

<i>Testing approach: Functionality testing, black box testing, REQ-STIM2.2, DR3, O2, S1</i>				
<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXPECTED RESULTS</i>	<i>COMMENTS</i>
1	Run <i>setup()</i> automatically or manually within the testing.ipynb file	Prepare Testing Data	Data loaded with no errors	
2	Run test_generate_networks_save_and_count() with 5 as a parameter	Generate specific amount of neural networks	Exactly 5 networks are created and	
Concluding Remarks: NA				
Testing Team: Leah Thomas,			Date Completed: 22 April 2025	

5.22 Test Case 22

Objective: Make sure max attempt threshold is enforced

Notes:

Test No.: 22			Current Status: Completed	
Test title: Verify max attempt threshold				
Testing approach: white box testing, REQ-F2, F4, Functionality testing				
STEP	OPERATOR ACTION	PURPOSE	EXEPTED RESULTS	COMMENTS
1	Run <i>setup()</i> automatically or manually within the testing.ipynb file	Prepare Testing Data	Data loaded with no errors	

2	Run <i>test_max_attempts_reached()</i> While generating networks	Validate Behavior when max training attempts are reached	Returns false if under maximum attempts and true if limit is met. Stopping network generation	
Concluding Remarks: NA				
Testing Team: Leah Thomas			Date Completed: 22 April 2025	

5.23 Test Case 23

Objective: Verify loss reset

Notes:

Test No.: 23			Current Status: Completed	
Test title: Loss Reset Validation				
Testing approach: white box testing REQ-STIM2.3, STIM2, Functionality Testing				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Run <i>setup()</i> automatically or manually within the testing.ipynb file	Prepare Testing Data	Data loaded with no errors	

2	Run <i>test_rest_loss()</i>	<i>Ensure loss histories are cleared after reset</i>	<i>Train_loss_history and test_loss_history are both empty lists</i>	
<i>Concluding Remarks: NA</i>				
<i>Testing Team: Leah Thomas</i>			<i>Date Completed: 22 April 2025</i>	